Resumen de teoría — ERP con Odoo (Unidades 1 y 2)

Apunte de estudio para el examen: abarca **todo el temario hasta la Unidad 2**.

1 — ERP y Odoo

Qué es un ERP

- **ERP** (*Enterprise Resource Planning*): sistema que gestiona **todas las áreas de la empresa de forma integral**.
- La información está **centralizada en una única base de datos**: no hay islas de información ni reingreso de datos entre sistemas.
- Las aplicaciones **se comunican entre sí**: registrar una venta actualiza inventario y dispara la factura. Esa **integración** es lo que diferencia a un ERP de varios sistemas sueltos.

Qué es Odoo

- ERP de **código abierto**, escrito principalmente en **Python** → curva de aprendizaje razonable para desarrolladores.
- Se despliega en la **nube** o en **servidores propios** (on-premise).
- **Una versión mayor por año** (mejoras de usabilidad, apps y procesos).
- Enfoque en **PyMEs**; al compararlo con SAP, ese punto es la diferencia: SAP apunta a grandes corporaciones con necesidades complejas.
- **Altamente configurable** (se adaptan procesos sin programar) y a la vez **modular y extensible**: apps del App Store, de terceros o desarrollo propio.

Community vs Enterprise

	Community	**Enterprise**
Costo	Gratuito	Licencia (suscripción)
Código	Abierto	Propietario sobre la base Community
Contenido	Módulos core (Ventas, Compras, Contabilidad, Inventario…)	Apps extra (Studio, reportes avanzados, IoT…) + Odoo.sh
Soporte	Comunidad	Oficial

La materia trabaja sobre **Community**.

Tecnologías y objetivo de la materia

- Stack: **Python** (lógica y modelos), **PostgreSQL** (base de datos), **XML** (vistas, acciones, menús, seguridad), **JS/OWL** (frontend).
- Objetivo del curso: decidir entre **configurar o desarrollar**, modelar con el **ORM**, implementar **vistas y lógica de negocio**, y levantar Odoo en contenedores con **Doodba**.

---

2 — Entorno de desarrollo: Doodba, pgweb y wdb

Doodba

- **Doodba Copier Template** (de Tecnativa): plantilla basada en **Copier** que genera un entorno de Odoo completo en **contenedores Docker**, con buenas prácticas (estructura, backups, herramientas de desarrollo).
- Objetivo de la materia: *tener una instancia de Odoo (en cualquier versión) con contenedores Docker usando Doodba*.
- El template incluye utilidades listas para usar: **pgweb** y **wdb**.
- Los módulos propios viven en `addons/` y se exponen a Odoo mediante symlinks en `private/` (Doodba solo lee addons desde ahí).

Puertos

Servicio	Guía (Odoo 18)	Entorno local (Odoo 19)
Odoo (web)	18069	19069
pgweb	18081	19081
wdb (depurador)	18984	19984

pgweb — inspeccionar la base de datos

- Interfaz web para navegar **PostgreSQL**: ver tablas, columnas, filas y correr SQL.
- Usos en la materia:
- Verificar que un campo existe como **columna real** en la tabla (o que no existe).
- Ver los **campos automáticos** que Odoo crea en toda tabla (`id`, `create_uid`, `create_date`, `write_uid`, `write_date`; detallados en el tema de modelos).

wdb — depurar la ejecución

- Depurador **web** que trae Doodba. En cualquier método se pone:

```
import wdb
wdb.set_trace()
```

- Al ejecutarse ese código, la corrida **se interrumpe** ahí. Desde el navegador (puerto del wdb), entrando al **hash de la sesión**: se ve el código, se inspeccionan y modifican variables, y se puede ejecutar código del ORM en vivo.

Modo debug y SuperUsuario

- **Modo desarrollador**: activa menús técnicos (modelos, vistas, campos, XML IDs) y utilidades como actualizar módulos desde Apps.
- **SuperUsuario** (usuario técnico `__system__`, **id = 1**; la cuenta `admin` es id 2 en una base estándar): **ignora todas las reglas de acceso**; ve y puede hacer todo. Sirve para probar sin depender de ACLs. Para **salir** del modo SuperUsuario: cerrar sesión y volver a identificarse.
- Consecuencia importante: con SuperUsuario **no aparecen errores de permisos**. Si algo "no se ve" o "no funciona", antes de asumir un bug conviene preguntarse si no es un problema de permisos y niveles de acceso.

---

3 — Anatomía de un módulo

El módulo: unidad básica de Odoo

Odoo es modular: **toda funcionalidad vive en un módulo**. Un módulo custom mínimo necesita **dos archivos raíz**:

- `__init__.py`: imports de Python del paquete (si hay carpeta `models/`, hace `from . import models`).
- `__manifest__.py`: diccionario con los **metadatos** del módulo.

Estructura de carpetas

```
real_estate/
├── __init__.py             # importa subpaquetes: from . import models
├── __manifest__.py         # metadatos: name, depends, data, license...
├── models/                 # clases Python que definen tablas
│   ├── __init__.py         # from . import estate_property, ...
│   └── estate_property.py
├── views/                  # XML: vistas, acciones y menús
├── security/               # grupos (XML) y reglas de acceso (ir.model.access.csv)
├── data/                   # datos cargados al instalar (valores iniciales)
├── demo/                   # datos de demostración (solo con instalación demo)
└── static/                 # imágenes, CSS, JS
```

Reglas de correspondencia:

- Los modelos van en `models/` y **deben estar importados**: primero en `models/__init__.py`, y ese paquete en el `__init__.py` del módulo. Si falta el import, la clase nunca se define y el modelo **no existe** para Odoo (no se ve en la base ni en la UI).
- Vistas → `views/`. Seguridad → `security/`.

El `__manifest__.py`

```
{
    "name": "Real Estate",
    "summary": "Real estate ads",
    "author": "UNLa",
    "version": "1.0",
    "depends": ["base"],                # módulos de los que depende
    "data": [                           # archivos cargados al instalar/actualizar
        "security/real_estate_res_groups.xml",
        "security/ir.model.access.csv",
        "views/estate_property_views.xml",
        "views/real_estate_menuitem.xml", # SIEMPRE al final
    ],
    "license": "LGPL-3",
    "installable": True,
    "application": True,
}
```

Claves que hay que saber explicar:

- **`depends`**: módulos que deben estar instalados antes. Odoo los instala automáticamente. Ej.: un módulo `estate_account` depende de `["real_estate", "account"]`.

- `data`: archivos XML/CSV que Odoo carga **en orden** al instalar y al **actualizar**.
- ⚠ **El orden importa** porque los archivos se referencian entre sí:
  1. los **grupos** antes del **CSV de accesos** (el CSV usa los XML ID de los grupos);
  2. las **vistas/acciones** antes que los **menús** (un menú referencia la acción por su XML ID). Si el orden se invierte, la actualización falla con "*External ID not found*": el registro referenciado todavía no existe.
- `demo`: datos de prueba, solo se cargan si la base se instala con datos demo.
- `application=True`: el módulo aparece como aplicación en el menú (no confundir con `installable`, que solo habilita que se pueda instalar).
- `license`: `LGPL-3` para módulos sobre Community.

Instalar vs actualizar

- **Instalar**: crea el módulo, carga sus datos y ejecuta su lógica inicial. Un módulo **nuevo** aparece en Apps recién después de *Update Apps List* (en modo debug).
- **Actualizar** (`-u módulo` o botón *Actualizar* en Apps): **relee el `data` completo** y actualiza lo definido en XML. Regla práctica: **cambios en modelos** (nuevos campos, atributos) y **cambios en XML** requieren actualizar el módulo para verse. Si un cambio de menú/vista "no aparece": actualizar, y en último caso limpiar caché del navegador.

XML ID (identificador externo)

- Todo registro creado desde XML recibe un **XML ID** con forma `modulo.prefijo_nombre` (ej.: `real_estate.action_estate_property`).
- Los registros se referencian entre archivos con `ref="xml_id"` (menú → acción, CSV → grupo, vista heredada → vista base).
- Para cada modelo Odoo genera automáticamente el XML ID `model_` + **nombre_del_modelo_con_guiones_bajos** (`estate.property` → `model_estate_property`); es el que se usa en la columna `model_id` del CSV de accesos.

---

4 — Modelos y campos

Definir un modelo

```
from odoo import api, fields, models

class EstateProperty(models.Model):
    _name = "estate.property"          # nombre técnico = tabla en PostgreSQL
    _description = "Propiedad"         # descripción humana (recomendado siempre)
```

- `_name` usa **puntos**; en la base la tabla se llama igual con **guiones bajos** (`estate.property` → tabla `estate_property`).
- `_description` es buena práctica obligatoria en la materia: la usan la UI, los logs y los mensajes de Odoo.
- Otros atributos de clase posibles: `_order` (orden por defecto de los listados) y `_rec_name` (qué campo se muestra como nombre del registro).

Tipos de campos simples

Tipo	Uso	Ejemplo del dominio
`Char`	texto corto (una línea)	`name`, `postcode`
`Text`	texto largo (varias líneas)	`description`
`Integer`	entero	`bedrooms`, `living_area`
`Float`	decimal	`expected_price`
`Monetary`	dinero con moneda (usa `currency_field`)	—
`Boolean`	sí/no	`garage`, `garden`
`Date`	fecha	`date_availability`
`Datetime`	fecha y hora	`create_date`
`Selection`	conjunto fijo de opciones	`garden_orientation`, `state`
`Binary`	archivos	adjuntos

Los campos de **relación** (`Many2one`, `One2many`, `Many2many`) se tratan en el Tema 6.

Atributos comunes de un campo

- **`string`**: etiqueta visible en la UI.
- **`required=True`**: obligatorio en la UI **y** columna `NOT NULL` en la tabla.
- **`default`**: valor inicial. Puede ser un literal (`default=2`, `default="north"`) o una **función/lambda** que se evalúa en el momento: `default=lambda self: self.env.user` (usuario logueado) o `default=lambda self: fields.Date.add(fields.Date.today(), months=3)` (fecha actual + 3 meses).
- **`copy=False`**: el campo **no se copia al duplicar** el registro (Acción → Duplicar). Criterio: un duplicado es un anuncio nuevo, no debe heredar datos de cierre: `date_availability`, `selling_price`, `state` y `salesman_id` llevan `copy=False`.
- **`readonly=True`**: solo lectura en la UI (además de los readonly "de nacimiento").
- **`help`**: tooltip. **`index=True`**: crea índice en BD (búsquedas más rápidas).
- **`selection=[('clave', 'Etiqueta'), ...]`**: la **clave** es lo que se guarda ("north"); la **etiqueta** es lo que se muestra ("Norte").

El campo `state` (convención)

- `state` es un nombre **convencional/reservado** en Odoo: representa el **flujo** del registro. Se define como `Selection`, `required=True`, un `default` inicial (`"new"`) y `copy=False`.
- En la vista formulario se muestra en el `<header>` con **`widget="statusbar"`** (la barra de estados que muestra el progreso del flujo).

Campos automáticos

Odoo crea en **toda** tabla, además de los campos definidos, columnas de auditoría:

- `id`: clave primaria.
- `create_uid` / `create_date`: quién y cuándo creó el registro.
- `write_uid` / `write_date`: quién y cuándo lo modificó por última vez.

Son los "campos adicionales" que aparecen al inspeccionar una tabla con pgweb.

Además, el ORM expone **campos mágicos** que no son columnas, como `display_name` (nombre legible del registro; en general el `name`): por eso **no aparece en pgweb**.

Convenciones de nombres

- Campos de relación Many2one → sufijo **`_id`** (`buyer_id`).
- One2many y Many2many → sufijo **`_ids`** (`offer_ids`, `tag_ids`).
- Métodos de cómputo → prefijo **`_compute_`**.
- Nombres técnicos en **inglés snake_case**; las etiquetas `string` en el idioma de la UI.

---

5 — Seguridad: usuarios, grupos y ACL

Tipos de usuario

Tipo	Grupo base	Quién es
Interno (backend)	`base.group_user`	Empleados que usan la interfaz de Odoo, con permisos según sus grupos
Portal	`base.group_portal`	Externos (clientes/proveedores) con acceso limitado vía portal web
Público	`base.group_public`	Visitantes sin login

Además existe el **SuperUsuario** (`__system__`, id = 1), que **ignora las reglas de acceso** (Tema 2).

Grupos (`res.groups`)

- Un **grupo** es el contenedor de permisos: accesos por modelo (ACLs), reglas de registro y visibilidad de menús/vistas/campos.
- Se pueden crear **desde la UI** (Ajustes → Usuarios → Grupos) o **formalizados en el módulo** (XML en `security/`):

```
<record id="group_estate_salesman" model="res.groups">
    <field name="name">Vendedor de Propiedades</field>
    <field name="category_id" ref="module_category_real_estate"/>
</record>
<!-- versión equivalente con id group_estate_manager y name "Manager de Propiedades" -->
```

> Versión: en **Odoo 18** (guía) los grupos se clasifican con `category_id` → `ir.module.category`. En **Odoo 19** (entorno local) el campo es `privilege_id` (modelo `res.groups.privilege`).

- **Categoría de aplicación**: agrupa los grupos del módulo del resto de los grupos nativos, para ubicarlos rápido en la vista de grupos (en la UI es el desplegable "Aplicación" del grupo).
- **`implied_ids`** (grupos implícitos): hace que un grupo incluya a otro; si Manager implica Vendedor, todo Manager hereda automáticamente también los permisos de Vendedor:

```
<field name="implied_ids" eval="[(4, ref('group_estate_salesman'))]"/>
```

Crear grupos desde la UI vs desde el módulo

Desde la UI	Desde el módulo (XML)
Rápido e inmediato, bueno para probar	**Reproducible y versionable** (git)
Se pierde al recrear/reinstalar la base	Se carga solo al instalar/actualizar el módulo
No viaja a otros entornos (dev → prod)	Mismo resultado en cualquier entorno

Desde la UI	Desde el módulo (XML)
Cambios no rastreables	Requiere actualizar el módulo

La forma correcta para producción es **definirlos en el módulo**.

ACL: `ir.model.access.csv`

Archivo en `security/`, incluido en el manifest, con una fila por grupo/modelo:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_estate_property_manager,estate.property.manager,model_estate_property,real_estate.group_es
access_estate_property_salesman,estate.property.salesman,model_estate_property,real_estate.group_
```

Cómo leer cada columna:

- `id` / `name`: identificador y descripción de la regla (libres).
- `model_id:id`: el modelo en cuestión → XML ID automático `model_` + nombre con guiones bajos (`model_estate_property`).
- `group_id:id`: grupo al que aplica (XML ID del grupo).
- `perm_read / perm_write / perm_create / perm_unlink`: `1/0` (CRUD).

Comportamiento clave:

- **Acumulativo**: un usuario pertenece a varios grupos → sus filas se **suman**.
- Un usuario **sin ningún grupo con permisos** sobre el modelo **no tiene acceso ninguno**: al intentar leer, Odoo lanza **AccessError**; tampoco le aparecen los menús ni las acciones de ese modelo.
- Una fila **sin `group_id`** aplica a **TODOS** los usuarios (incluidos portal y público): situación peligrosa a evitar.
- Modelo nuevo sin reglas → nadie (salvo SuperUsuario) puede usarlo. Preguntarse "*¿tengo permisos?*" antes de asumir que algo se rompió.

Otras capas de seguridad

- **ACL** (este tema): controla el acceso al **modelo completo** (CRUD).
- **Reglas de registro (`ir.rule`)**: filtran **filas** dentro del modelo (ej.: "solo ve sus propios registros" por usuario o compañía).
- **`groups="..."` en vistas/menús/campos**: **oculta** elementos de la interfaz a quien no pertenece al grupo. Es cosmética: mejora UX, **no reemplaza** la validación en el servidor (el servidor debe seguir negando la operación).

---

6 — Relaciones entre modelos

Many2one (muchos de acá → uno de allá)

```
buyer_id = fields.Many2one(comodel_name="res.partner", string="Comprador")
salesman_id = fields.Many2one(
    comodel_name="res.users", string="Vendedor",
    copy=False, default=lambda self: self.env.user,
)
property_type_id = fields.Many2one(
    comodel_name="estate.property.type", string="Tipo Propiedad",
)
```

- Guarda la **foreign key en la tabla de este modelo**: `buyer_id` → `res_partner.id`; `salesman_id` → `res_users.id`.
- `comodel_name` declara el modelo destino.
- Es la relación por defecto cuando cada registro apunta a **un solo** registro del otro modelo: el vendedor de una propiedad, su comprador, su tipo.

One2many (uno de acá → muchos de allá)

```
offer_ids = fields.One2many(
    comodel_name="estate.property.offer",
    inverse_name="property_id",
    string="Ofertas",
)
```

- **No crea tabla ni columna**: es **virtual**. Odoo la resuelve buscando en el modelo hijo los registros cuyo `Many2one` apunta hacia acá.
- **`inverse_name` es obligatorio**: es el nombre del **Many2one del hijo** que "vuelve" hacia este modelo. Sin él, Odoo no sabe qué campo filtrar en el hijo.
- La información vive **en la tabla del hijo** (la FK está en `estate_property_offer.property_id`).
- Forma abreviada equivalente: `fields.One2many("estate.property.offer", "property_id", string="Ofertas")`.
- Regla del dúo clásico: **el mismo par de modelos se declara Many2one en el hijo y One2many en el padre** (la oferta tiene `property_id`; la propiedad tiene `offer_ids`).

¿Por qué la One2many es virtual? Porque es solo la vista inversa de un Many2one que ya existe en el hijo; no agrega estructura propia. Consecuencia: la asignación de sus registros se hace con **comandos relacionales** (Tema 11).

Many2many (muchos ↔ muchos)

```
tag_ids = fields.Many2many(comodel_name="estate.property.tag", string="Etiquetas")
```

- Odoo crea una **tabla intermedia** que guarda los pares (propiedad, etiqueta).
- Ambos lados pueden tener muchos: una propiedad tiene varias etiquetas y una etiqueta puede estar en varias propiedades.

¿Por qué etiquetas es Many2many y no Many2one ni One2many?

- **Many2one** de propiedad → etiqueta solo permitiría **una etiqueta por propiedad**.

- **One2many** pondría la FK **en la etiqueta**: cada etiqueta pertenecería a **una sola propiedad** (imposible: la misma etiqueta "Luminosa" debe servir para varias).
- **Many2many** reproduce la relación **N:N** real.

Cómo elegir la relación

Situación	Relación	Dónde se guarda
Cada registro apunta a **1** del otro modelo	Many2one	columna FK **en esta tabla**
Un registro contiene a **muchos** del otro (FK en el hijo)	One2many	nada propio: FK **en el hijo**
Muchos ↔ muchos	Many2many	**tabla intermedia**

---

7 — Vistas, acciones y menús

Tipos de vistas y comportamientos básicos

Tipo	Tag XML	Para qué
Lista	`<list>`	columnas, orden, edición en línea
Formulario	`<form>`	alta/edición de un registro
Búsqueda	`<search>`	buscar, filtrar, agrupar
Kanban	`<kanban>`	tarjetas

- Si un modelo **no define** su vista lista/formulario, Odoo muestra una **genérica** (una lista con solo el nombre). Al crearse una vista propia del modelo, se usa automáticamente como principal.
- Las vistas se definen como registros `ir.ui.view` con `model` (modelo al que aplican), `inherit_id` opcional (herencia, Tema 14) y el XML dentro de `<field name="arch" type="xml">`.

Acción (`ir.actions.act_window`)

```
<record id="action_estate_property" model="ir.actions.act_window">
    <field name="name">Propiedades</field>
    <field name="res_model">estate.property</field>
    <field name="view_mode">list,form</field>
</record>
```

- La acción es el **puente entre menú y modelo**: dice qué modelo abrir y con qué vistas.
- **`view_mode`** define qué vistas hay disponibles y **el orden importa**: se muestra primero la primera.
- `list,form`: abre el listado; desde ahí cada registro abre su formulario. Es el estándar.
- Solo `list`: se ven los registros pero **no se puede abrir un registro** (no hay formulario al que entrar).
- Solo `form`: la acción abre **directamente un formulario** (sin listado previo).
- Otras claves posibles: `domain` (filtro fijo), `context`, `target` (`new` abre ventana emergente).

Menús (`ir.ui.menu`)

```
<menuitem id="menu_estate_root" name="Inmobiliaria"/>
<menuitem id="menu_estate_ads" name="Anuncios" parent="menu_estate_root"/>
<menuitem
    id="menu_estate_property"
    name="Propiedades"
    parent="menu_estate_ads"
    action="action_estate_property"/>
```

- Estructura de **tres niveles**: raíz (barra principal) → primer nivel (desplegable) → menú **de acción** (`action="xml_id_de_la_accion"`).
- Un menú **sin `action` no hace nada** al clickearlo (solo sirve de agrupador).
- El archivo de menús debe cargarse **después** de las vistas/acciones en el manifest: si no, el menú hace `ref` a una acción que **todavía no existe** y la actualización falla con "*External ID not found*".
- **Visibilidad:** si el usuario **no tiene permisos** sobre el modelo de la acción, el menú **no aparece** (Odoo lo oculta al no poder usarse). Por eso un módulo recién instalado, todavía sin reglas de acceso, no muestra sus menús: es permisos, no un bug.

Vista lista

```
<list>
    <field name="name"/>
    <field name="postcode"/>
    <field name="tag_ids" widget="many2many_tags"/>
</list>
```

- **`editable="top"`** (o `bottom`): edición/creación **en línea** sin abrir formulario. Cómodo para modelos de pocos campos (etiquetas, tipos). Cuando crear el registro requiere más lógica (campos que se autocompletan, validaciones), conviene crear desde el formulario.
- **`widget="many2many_tags"`**: muestra las relaciones m2m como chips.

Vista formulario

Estructura de referencia:

```
<form>
    <header>
        <button name="action_cancel" string="Cancelar" type="object"
                invisible="state not in ['new', 'offer_received']"/>
        <button name="action_sold" string="Marcar como vendida" type="object"
                invisible="state in ['sold', 'canceled']"/>
        <field name="state" widget="statusbar"/>
    </header>
    <sheet>
        <widget name="web_ribbon" title="Vendida" bg_color="bg-success"
                invisible="state != 'sold'"/>
        <h1><field name="name" placeholder="Nombre propiedad"/></h1>
        <field name="tag_ids" widget="many2many_tags"/>
        <group>
            <group><!-- columna 1 --></group>
            <group><!-- columna 2 --></group>
        </group>
        <notebook>
            <page string="Descripción">…</page>
            <page string="Ofertas"><field name="offer_ids"/></page>
        </notebook>
    </sheet>
</form>
```

Componentes que hay que poder nombrar:

- **`<header>`**: botones de flujo y el campo `state` con `widget="statusbar"`.
- **`<sheet>`**: la "hoja" del formulario (todo el resto va adentro).
- **`<group>`**: grilla de campos; **dos `<group>` hermanos = dos columnas**.
- **`<notebook>`/`<page>`**: pestañas.
- **`placeholder`**: texto guía dentro del campo vacío.
- **Ribbon**: etiqueta de esquina (`web_ribbon`) con color (`bg-success`, `bg-warning`…).
- **Botones**: `type="object"` llama a un **método del modelo** (`name="action_sold"`); `type="action"` dispara otra acción de Odoo.
- **`invisible="..."`**: expresión estilo Python sobre los campos de la vista (Odoo 17+; ya no se usa el viejo `attrs`). Controlar la visibilidad según estado es UX; la **validación** fuerte va en el servidor (Tema 12).
- **`groups="grupo_xml_id"`** en campos/botones/menús: visible solo para ese grupo.

Vista search

```
<search>
    <field name="name"/>
    <field name="postcode"/>
    <filter name="my_properties" string="Mis propiedades"
            domain="[('create_uid', '=', uid)]"/>
    <filter name="available" string="Disponibles"
            domain="[('state', 'in', ['new', 'offer_received'])]"/>
    <group>
        <filter name="groupby_state" string="Estado"
                context="{'group_by': 'state'}"/>
        <filter name="groupby_month" string="Mes de creación"
                context="{'group_by': 'create_date:month'}"/>
    </group>
</search>
```

- **`<field name="...">`**: agrega el campo como **término de búsqueda** del search bar.
- **`<filter domain="...">`**: filtro guardado (chips) sobre un dominio. El usuario actual se referencia con **`uid`** dentro del dominio o contexto.
- **Agrupar por**: filtro con `context="{'group_by': 'campo'}"`, anidado en `<group>` para aparecer en el menú "Agrupar por". Sufijo `:month` para agrupar por mes de una fecha (`create_date:month`).

Vistas embebidas: cuando una lista no necesita acción

- Una vista es una **plantilla reutilizable**. La vista lista de ofertas, por ejemplo, no se abre desde un menú sino que se muestra **embebida** dentro del formulario de la propiedad (campo `offer_ids`).
- Una **acción + menú propios** solo se necesitan cuando se quiere abrir el modelo de forma independiente (ej.: un listado de "todas las ofertas" desde el menú).

---

8 — Campos computados, related e inverse

Campo computado

Campo cuyo valor **no se ingresa: se calcula** por un método:

```
total_area = fields.Float(string="Superficie total", compute="_compute_total_area")

@api.depends("living_area", "garden_area")
def _compute_total_area(self):
    for record in self:
        record.total_area = record.living_area + record.garden_area
```

Reglas:

- `compute` nombra el método; convención: prefijo **`_compute_`**.
- El método **siempre itera** `for record in self`: `self` es un **recordset** que puede traer uno o muchos registros (una vista lista dispara el cálculo para todos).
- Por defecto es **solo lectura**: la UI no lo deja editar.

Dónde vive el valor: `store`

	Sin `store` (por defecto)	**Con `store=True`**
¿Columna en la tabla (pgweb)?	**No**	**Sí**
¿Cuándo se calcula?	Cada vez que una vista lo muestra	Una vez (al crear/modificar), luego se **lee de la base**
¿Buscar / agrupar / ordenar por él?	**No** (no hay columna)	**Sí**
¿Se copia al duplicar el registro?	No (los computados usan `copy=False`)	No (salvo `copy=True` explícito)

La lógica: sin `store`, el valor vive en memoria y se recalcula en cada render; con `store`, se guarda el valor ya calculado para la próxima vez **leer** la base y ahorrar el costo del cómputo (pagando con espacio en disco).

¿Cuándo recalcular y cuándo guardar? (`@api.depends`, `store`)

- **`store=True` sin `@api.depends`**: el valor se calcula al crear y nunca más se recalcula → queda **desactualizado**: cambiaron `living_area`/`garden_area` y no pasa nada. Motivo: el método ya corrió, Odoo **no sabe** a qué campos debe estar atento para dispararlo de nuevo.
- **`@api.depends("living_area", "garden_area")`** registra las dependencias: al escribir en cualquiera de esos campos, Odoo **reprograma el cómputo** del campo almacenado y el valor vuelve a actualizarse.
- Con campos **no almacenados**, `depends` también sirve: mantiene fresco el valor en la vista mientras arriba cambiaron los campos que lo definen.
- `depends` admite **rutas**: `@api.depends("offer_ids.price")` declara dependencia del precio de las ofertas del registro (no de las ofertas en sí).

> **`store` define dónde vive el valor (bd o memoria); `depends` define a qué Odoo debe prestar atención para recalcularlo.** Con store y sin depends: se congela el valor.

Criterios para almacenar o no

- Se muestra solo en **formulario** (un registro a la vez) → no hace falta almacenar.
- Se necesita para **buscar/agrupar/ordenar** → almacenar (o implementar `_search`; por simplicidad: almacenar).
- Aparece en **listas** (pueden verse cientos de filas por vez) → conviene almacenar.
- El cálculo **recorre/agrega datos de otro modelo** (ej. `best_offer` sobre las ofertas) → **no almacenar**, salvo que se necesite buscar/agrupar/ordenar; evita recomputos encadenados. Caso `best_offer` (máximo de las ofertas): no almacenado, con `depends("offer_ids.price")`, es suficiente:

```
best_offer = fields.Float(string="Mejor oferta", compute="_compute_best_offer")

@api.depends("offer_ids.price")
def _compute_best_offer(self):
    for record in self:
        record.best_offer = max(record.offer_ids.mapped("price"), default=0)
```

- ⚠ Peligro real: un computado almacenado que depende de un campo de otro modelo (ej.: `@api.depends("partner_id.name")` con `store=True`) obliga a recomputar **todos** los registros que referencian a ese partner cuando el partner cambia. Con volumen de datos, el sistema se vuelve

lento. El `related` almacenado es la excepción práctica: se usa para agrupar y se asume ese costo (ver más abajo).

Campo related

```
partner_id = fields.Many2one("res.partner", string="Ofertante")
email = fields.Char(related="partner_id.email")
# en la oferta (related almacenado):
property_type_id = fields.Many2one(
    related="property_id.property_type_id",
    store=True,
    string="Tipo de propiedad",
)
```

- Trae el valor **siguiendo una cadena de relaciones** (`property_id.property_type_id`).
- Concepto: un related es **un campo computado simplificado**: implementa solo un caso (leer un campo de otro registro). Evita escribir un `compute` trivial.
- Por defecto: **no se almacena** y es **readonly**.
- Con **`store=True`**: la columna existe → queda disponible para **buscar y agrupar** (por ejemplo: agrupar el listado de ofertas por tipo de propiedad de su propiedad). El valor se recomputa solo (dependencia implícita en la cadena).
- Cuidar los tipos: el tipo de dato del related debe **coincidir** con el del campo original.

¿Un related sirve para traer "todos los ofertantes" de una propiedad? No: el related implica **un** camino fijo a **un** valor, y acá hay que juntar los partners de **N** ofertas. Eso se resuelve con un **Many2many computado**:

```
offer_partner_ids = fields.Many2many("res.partner", compute="_compute_offer_partner_ids")

@api.depends("offer_ids.partner_id")
def _compute_offer_partner_ids(self):
    for record in self:
        record.offer_partner_ids = record.offer_ids.partner_id
```

Computed editable: `inverse`

```
doble_de_numero = fields.Integer(
    string="Doble de número",
    compute="_compute_doble",
    inverse="_inverse_doble",   # ← hace el campo escribible
)

def _compute_doble(self):
    for rec in self:
        rec.doble_de_numero = rec.numero * 2

def _inverse_doble(self):
    for rec in self:
        rec.numero = rec.doble_de_numero / 2
```

- La sola presencia de **`inverse=`** hace **escribible** el campo computado: cuando el usuario ingresa un valor, Odoo llama al método inverso para **devolver** el efecto al campo fuente.

- Caso de dominio: `date_deadline = create_date + validity` (computa la fecha límite); con el inverse, si el usuario escribe la fecha límite, se recalcula `validity = date_deadline - create_date`. Si el registro todavía no existe (`create_date` vacío), se usa la fecha de hoy como base.

Tabla resumen (memorizar esta)

Variante	¿Vive en bd?	¿Se recalcula al cambiar el origen?	¿Buscar/agrupar?	¿Escribible?
`compute` sin store	no	sí (en cada render)	no	no
`compute` + `store` (sin depends)	sí	**no** (queda congelado)	sí	no
`compute` + `store` + `depends`	sí	sí	sí	no
`compute` + `inverse`	no (salvo store)	sí	no	**sí**
`related`	no (salvo `store=True`)	sí (dependencia automática)	solo si `store=True`	no

---

9 — `onchange`: lógica en el formulario

Qué es y cuándo corre

```
@api.onchange("garden")
def _onchange_garden(self):
    if self.garden:
        self.garden_area = 10
    else:
        self.garden_area = 0
```

- Se **dispara desde el formulario** (cliente) **mientras se edita** (antes de guardar), cuando cambia alguno de los campos listados en el decorador; el método Python corre en el servidor.
- Permite **modificar otros campos del registro en pantalla** (autocompletar, sugerir valores): si el usuario guarda, van con el registro; si cancela, se descartan.
- `self` es un único registro (el del formulario).

Límites (importante)

- **No persiste nada por sí mismo**: es una ayuda al tipeo, no un dato garantizado.
- **No corre en escrituras desde código** (métodos del ORM, imports, XML-RPC): ahí no hay formulario que dispare el evento.
- Por eso **no sirve como validación de negocio ni control de seguridad**: para bloquear una operación hay mecanismos de servidor (Tema 12). `onchange` es **UX**, las validaciones fuertes van en el servidor.

Aviso no bloqueante (`warning`)

```
@api.onchange("expected_price")
def _onchange_expected_price(self):
    if self.expected_price and self.expected_price < 10000:
        return {
            "warning": {
                "title": "Precio bajo",
                "message": "El precio esperado es menor a 10000, posible error de tipeo.",
            }
        }
```

- Devolver `warning` muestra un cartel al usuario pero **no impide guardar**.
- Punto de depuración típico: poner `wdb.set_trace()` dentro del onchange y mirar la ejecución en el wdb al editar el campo.

---

10 — ORM y CRUD

El ORM

- Odoo expone un **ORM** en Python: los modelos son clases, los registros son **objetos** (recordsets) y todo CRUD se escribe en Python **sin SQL manual** (el ORM lo traduce a SQL sobre PostgreSQL).

`env` y recordsets

- Todo acceso arranca por el **environment** (`self.env`):

```
self.env["estate.property"]          # acceso al modelo (recordset vacío)
self.env["estate.property"].search([...])  # ejecuta → devuelve recordset
self.env.user                        # usuario actual
self.env.context                     # contexto actual (dict)
self.env.su                          # flag de superusuario
```

- **Recordset**: colección **inmutable** de registros (cero, uno o muchos) que a la vez es acceso al modelo. Antes de tocar campos se **itera**: `for record in self: ...` (es el patrón estándar de casi cualquier método).
- Herramientas sobre recordsets:
- `rec.id` (cuando es uno) / `rec.ids` (lista de ids) / `rec.display_name`.
- `ensure_one()`: exige que sea exactamente un registro (o falla).
- `mapped("campo")`: lista/recordset con ese campo de cada registro (`self.offer_ids.mapped("price")`).
- `filtered(...)` y `sorted(...)`: filtrar/ordenar en Python.
- `browse(id)`: construir un recordset a partir de ids.
- `exists()` / `copy()`.
- Operadores: `rs1 + rs2` (unión) y `rs1 - rs2` (diferencia; ej.: listar todas las ofertas salvo una: `property.offer_ids - offer`).

CRUD

```
# CREATE
prop = self.env["estate.property"].create({"name": "Casa", "bedrooms": 3})

# READ / SEARCH (dominios)
recs = self.env["estate.property"].search(
    [("state", "=", "new"), ("expected_price", ">", 100000)],
    limit=10,
    order="expected_price desc",
)
uno = self.env["estate.property"].browse(5)          # ubicar por id
n = self.env["estate.property"].search_count([...])  # contar sin traer registros

# UPDATE (aplica el MISMO dict a todos los registros del recordset)
recs.write({"state": "canceled"})

# DELETE
prop.unlink()
```

Dominios

- Un dominio es una lista de tuplas `(campo, operador, valor)`.
- Combinación por defecto: **Y lógico** entre todas las tuplas. Operadores de prefijo para combinar a mano: `&` (y), `|` (o), `!` (no).
- Operadores comunes: `=` `!=` `<` `<=` `>` `>=` `like` `ilike` `in` `not in`.
- Ejemplos:
- "creados por el usuario actual": `[('create_uid', '=', uid)]`
- "disponibles": `[('state', 'in', ['new', 'offer_received'])]`
- "contactos activos que no ofertaron": `[('active', '=', True), ('id', 'not in', offered_ids)]`

Redefinir `create()` — el patrón

En versiones modernas, `create` se declara con `@api.model_create_multi` y recibe **una lista de dicts** (creación en lote). Patrón completo:

```
@api.model_create_multi
def create(self, vals_list):
    # 1) validar ANTES de crear cada vals (raise UserError si no cumple)
    for vals in vals_list:
        # ej.: price debe superar best_offer; propiedad en estado válido
        ...
    # 2) llamar al super (no se reimplementa el INSERT)
    records = super().create(vals_list)
    # 3) efectos posteriores a la creación
    records.property_id.write({"state": "offer_received"})
    # 4) devolver SIEMPRE lo que devuelve super
    return records
```

Redefinir `write()` sigue la misma idea: validar, llamar `super().write(vals)`, efectos, devolver. La clave conceptual: **extender, no reemplazar**: la lógica original del ORM sigue corriendo.

---

11 — Comandos relacionales

Qué son

Para **agregar/quitar/actualizar registros de un campo One2many o Many2many** no se asignan ids "a mano": se les asigna una **lista de comandos**, internamente tuplas de **tres elementos**:

1. un número que identifica la operación,
2. el id del registro (si aplica),
3. un dict con valores (si aplica).

Los 7 comandos

N°	Nombre	Tupla	Qué hace
0	CREATE	`(0, 0, valores)`	**crea y vincula** un registro nuevo
1	UPDATE	`(1, id, valores)`	**actualiza** un registro ya vinculado
2	DELETE	`(2, id, 0)`	**borra el registro** de la BD y rompe el vínculo
3	UNLINK	`(3, id, 0)`	solo **desvincula**; el registro **sigue existiendo**
4	LINK	`(4, id, 0)`	**vincula** un registro que ya existe
5	CLEAR	`(5, 0, 0)`	**desvincula todos** (vacía la relación)
6	SET	`(6, 0, ids)`	reemplaza la relación por **exactamente** esa lista de ids

Helpers `Command`

Los helpers de `from odoo import Command` se corresponden 1:1 con las tuplas (y son la forma recomendada, más legible; las tuplas crudas también son válidas): `Command.create(values)`, `Command.update(id, values)`, `Command.delete(id)`, `Command.unlink(id)`, `Command.link(id)`, `Command.clear()`, `Command.set([ids])`.

Diferencias que toman examen

- **`unlink` vs `delete`**: `unlink` saca la etiqueta **de esta relación**, pero el registro sigue en su tabla (puede vincularse a otro). `delete` además **lo elimina** de la base.
- **`clear` vs `set`**: `clear` desvincula **todo**; `set` deja vinculados **solo** los de la lista (los anteriores que no estén en la lista, desvinculados).
- En **One2many**, `Command.create` usa el `inverse_name`: el hijo se crea con la FK asignada automáticamente hacia el padre.

Ejemplos con el dominio

```
# Vaciar todas las etiquetas
record.tag_ids = [Command.clear()]

# Dejar exactamente todas las etiquetas existentes
record.tag_ids = [Command.set(all_tags.ids)]

# Vincular una etiqueta concreta (creada si no existía)
record.tag_ids = [Command.link(tag.id)]

# Patrón "A estrenar": buscar, crear si falta y vincular
tag = self.env["estate.property.tag"].search([("name", "=", "A estrenar")], limit=1)
if not tag:
    tag = self.env["estate.property.tag"].create({"name": "A estrenar"})
record.tag_ids = [Command.link(tag.id)]

# Crear una oferta (registro hijo) ya vinculada a la propiedad
property.offer_ids = [
    Command.create({"price": 120_000, "partner_id": partner.id}),
]
```

---

12 — Validaciones y control de errores

`_sql_constraints`: restricciones en la base

Atributo de clase: lista de tuplas **(nombre, definición SQL, mensaje de error)**:

```
_sql_constraints = [
    ("unique_tag_name", "UNIQUE(name)", "El nombre de la etiqueta debe ser único"),
    ("partner_property_uniq",
     "UNIQUE(partner_id, property_id)",
     "Un ofertante no puede ofertar dos veces por la misma propiedad."),
]
```

- **`UNIQUE(col)`**: unicidad de una columna.
- **`UNIQUE(col1, col2)`**: unicidad **compuesta**: prohibe repetir la **combinación**; cada columna por separado puede repetirse (un cliente sí puede ofertar en varias propiedades, pero **no dos veces** por la misma).
- **`CHECK(expresión)`**: condición arbitraria en SQL.
- Las validaciones las aplica **PostgreSQL**, por lo tanto **siempre se cumplen**, hasta si alguien escribe con SQL directo. Al violarse, Odoo muestra el **mensaje** de la tercera posición de la tupla.
- ⚠ Antes de agregar un `UNIQUE`, hay que **limpiar los datos duplicados existentes**; si no, la actualización del módulo falla (la restricción no puede crearse con datos que la violen).

`@api.constrains`: restricciones en Python

```
from odoo.exceptions import ValidationError

@api.constrains("price")
def _check_price(self):
    for rec in self:
        if rec.price < 0:
            raise ValidationError("El precio no puede ser negativo.")
```

- Corren en el **servidor** cada vez que el registro se crea o modifica **vía ORM**.
- Sirven para reglas que SQL no expresa (lógica multi-campo, llamar a otros modelos).
- Lanzan **`ValidationError`**.
- **No corren** si el cambio viene de SQL directo: ahí están las `_sql_constraints`.

`UserError`: reglas de negocio que abortan la operación

```
from odoo.exceptions import UserError

raise UserError("No se puede cancelar una propiedad vendida.")
```

- `raise UserError(...)` **detiene la operación** y muestra ese mensaje al usuario en un diálogo de error. Es la herramienta para reglas de negocio dentro de métodos y botones (flujo: cancelar, vender, crear una oferta inválida...).

Qué mecanismo uso para cada objetivo (una sola tabla, para todo)

Objetivo	Mecanismo	¿Bloquea?	Dónde corre
Avisar "posible error de tipeo" sin frenar	`onchange` que devuelve `warning`	no	servidor, disparado por el form
Sugerir / autocompletar valores	`onchange`	no	servidor, disparado por el form
Impedir una operación por regla de negocio	`raise UserError` en método/botón	**sí**	servidor
Validar datos al crear/modificar vía ORM	`@api.constrains` + `ValidationError`	**sí**	servidor (ORM)
Integridad dura y definitiva	`_sql_constraints`	**sí, siempre**	PostgreSQL

Escala conceptual: `onchange` solo informa < `@api.constrains` bloquea escrituras por ORM < `_sql_constraints` bloquea hasta a mano en SQL. `UserError` protege el **flujo** de negocio (botones y override de `create`/`write`).

---

13 — Lógica de negocio: botones, estados y borrado

Métodos de botón

- La vista los invoca con `type="object"`:

```
<button name="action_sold" string="Marcar como vendida" type="object"
        invisible="state in ['sold', 'canceled']"/>
```

- Convención de nombres: `action_<algo>`. El método recibe `self` (el registro del form o los seleccionados de la lista) y normalmente devuelve `None`.
- Doble defensa sobre el flujo: **validación en backend** (`UserError`) + **esconder el botón en frontend** según estado (`invisible`). El ribbon "Vendida" (`web_ribbon`) es solo feedback visual del estado.

Cambiar de estado con reglas

```
def action_sold(self):
    for record in self:
        if record.state == "canceled":
            raise UserError("No se puede vender una propiedad cancelada.")
        record.state = "sold"
```

Cada cambio de estado mezcla: validación (`UserError`) + actualización del campo `state` + efectos colaterales (ver "aceptar oferta" abajo).

`@api.ondelete`: proteger el borrado

```
@api.ondelete(at_uninstall=False)
def _unlink_if_new_or_cancelled(self):
    for record in self:
        if record.state not in ("new", "canceled"):
            raise UserError("Solo se pueden borrar propiedades Nuevas o Canceladas.")
```

¿Por qué `ondelete` y no redefinir `unlink()`?

1. El método **solo valida y lanza**: no hay que reimplementar el borrado ni llamar a `super().unlink()` (menos lugar para romper la operación real).
2. Es el **mecanismo que Odoo espera** para casos de este tipo: el framework llama al chequeo previo y maneja el borrado él solo.
3. **`at_uninstall=False`** permite que al **desinstalar el módulo** Odoo borre los registros igual; una validación incondicional dentro de un `unlink()` redefinido podría **romper la desinstalación**.
4. Redefinir `unlink()` intercepta **todos** los borrados del modelo, incluso los internos del framework; con `ondelete` el borrado queda en manos del framework y el método solo se limita a validar.

Redefinir `create()` en ofertas — flujo completo

```
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        prop = self.env["estate.property"].browse(vals["property_id"])
        # a) la propiedad debe estar en un estado válido
        if prop.state not in ("new", "offer_received"):
            raise UserError(
                "Solo se puede ofertar en propiedades Nuevas u Ofertas recibidas."
            )
        # b) la oferta debe superar la mejor oferta actual
        if vals["price"] <= prop.best_offer:
            raise UserError("La oferta debe superar la mejor oferta actual.")
    records = super().create(vals_list)
    # c) la propiedad pasa a "Oferta recibida"
    records.property_id.write({"state": "offer_received"})
    return records
```

La secuencia general de cualquier override de ORM: **validar → super → efectos → return**.

Aceptar una oferta: qué dispara en cascada

Botón en la lista de ofertas (`action_accept`):

```
def action_accept(self):
    for offer in self:
        offer.status = "accepted"
        property = offer.property_id
        property.buyer_id = offer.partner_id        # el ofertante pasa a comprador
        property.selling_price = offer.price        # precio de venta = precio ofertado
        property.state = "offer_accepted"           # flujo de la propiedad avanza
        (property.offer_ids - offer).write({"status": "refused"})  # el resto se rechaza
```

Puntos clave: **qué valores recibe la propiedad** (`buyer_id`, `selling_price`), **qué estado toma la propiedad** (deja de aceptar ofertas nuevas) y **qué pasa con las demás ofertas** (todas quedan rechazadas; la expresión usa el operador resta de recordsets).

Generar una oferta automática (búsqueda + azar + ORM)

Botón en el formulario de la propiedad que crea una oferta sobre contactos que todavía no ofertaron:

```
def action_generate_offer(self):
    import random

    for record in self:
        if not record.expected_price:
            raise UserError("La propiedad no tiene precio esperado.")
        candidates = self.env["res.partner"].search([
            ("active", "=", True),
            ("id", "not in", record.offer_partner_ids.ids),
        ])
        if not candidates:
            raise UserError("No quedan contactos disponibles para ofertar.")
        self.env["estate.property.offer"].create({
            "price": record.expected_price * random.uniform(0.7, 1.3),
            "partner_id": random.choice(candidates).id,
            "property_id": record.id,
        })
```

- Ejercita lo típico de un botón: **dominio** para filtrar candidatos, **M2M computado** para saber quién ya ofertó, `random` para precio/ofertante y **creación vía ORM**.
- Si la oferta generada no supera la mejor oferta, el `create()` redefinido (más arriba) la rechaza con `UserError`: la regla de negocio se respeta venga de donde venga.

---

14 — Herencia

Herencia de **modelos**, de **métodos**, de **vistas** y entre **módulos**. Es el mecanismo por el que Odoo extiende todo sin reescribir nada.

Modelos: `_inherit` sin `_name` (extender)

```
class ResUsers(models.Model):
    _inherit = "res.users"   # SIN _name → mismo modelo, misma tabla

    property_ids = fields.One2many(
        comodel_name="estate.property",
        inverse_name="salesman_id",
        string="Propiedades",
    )
```

- La clase nueva **se fusiona** con el modelo existente: agrega campos/métodos a la **misma tabla** (`res_users`).
- **¿Hacen falta reglas nuevas en `ir.model.access.csv`? No**: no se creó un modelo nuevo, sigue siendo `res.users`, que **ya tiene sus reglas de acceso**. (Recién haría falta si el `_inherit` viniera acompañado de un `_name`, es decir, un modelo nuevo.)

Modelos: `_inherit` con `_name` (copiar)

- `_name = "otro.modelo"` **y** `_inherit = "res.partner"` a la vez: crea un **modelo nuevo** (otra tabla) que **copia** campos y métodos del padre. No es el caso normal de extensión: es duplicar la estructura.

Modelos: `_inherits` (delegación)

- `_inherits = {"res.partner": "partner_id"}`: el hijo tiene un `Many2one` al padre y **expone los campos del padre como propios** (delegación). Es el patrón de `res.users` sobre `res.partner`.
- No se usa en los TP de estas unidades; hay que saber **no confundirla** con `_inherit`.

Métodos: `super()`

```
class EstateAccount(models.Model):
    _inherit = "estate.property"

    def action_sold(self):
        res = super().action_sold()   # 1) la lógica original corre primero
        # 2) además: generar la factura
        for record in self:
            if record.buyer_id and record.selling_price:
                self.env["account.move"].create({
                    "partner_id": record.buyer_id.id,
                    "move_type": "out_invoice",   # factura de venta a cliente
                    "invoice_line_ids": [
                        Command.create({"name": record.name, "quantity": 1,
                                        "price_unit": record.selling_price}),
                        Command.create({"name": "Gastos administrativos", "quantity": 1,
                                        "price_unit": 100}),
                    ],
                })
        return res
```

- La regla de los módulos que se apilan: **redefinir y llamar a `super()`** para no perder el comportamiento original.
- La factura se genera **solo si hay comprador y precio de venta**; si faltan, no se factura (la venta en sí sigue funcionando).
- `move_type`: `out_invoice` = factura de venta a cliente; `in_invoice` = compra a proveedor; `out_refund` = nota de crédito a cliente.

Vistas: herencia con `inherit_id` + `xpath`

```
<record id="res_users_view_form" model="ir.ui.view">
    <field name="name">res.users.form (real_estate)</field>
    <field name="model">res.users</field>
    <field name="inherit_id" ref="base.view_users_form"/>
    <field name="arch" type="xml">
        <xpath expr="//notebook" position="inside">
            <page string="Propiedades">
                <field name="property_ids" readonly="1"/>
            </page>
        </xpath>
    </field>
</record>
```

- **`inherit_id`** apunta a la vista base (nativa o de otro módulo).
- **`xpath`**: localiza el nodo de la vista original; **`position`** dice qué hacer: `inside`, `after`, `before`, `replace`, `attributes`…

- La gracia: **no se reescribe la vista original**, solo se agrega. Si el módulo nativo cambia, la extensión sigue aplicando.

Módulos: dependencias

```
# estate_account/__manifest__.py
{
    "name": "Estate Account",
    "depends": ["real_estate", "account"],
}
```

- Con `depends`, Odoo instala primero las dependencias, y el módulo nuevo puede **extender sus modelos** (herencia de clases Python sobre addons ya existentes).
- Es el patrón arquitectónico de Odoo: **addons extienden otros addons**.